

BATCH 03

TASK 01 : APPLICATION OF N-GRAM MODEL TO THE TEXT

```
import nltk
import collections
import math
import string
import pandas as pd
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
text=["Iam working as a professor.",
      "Iam working in SR University."]
tokens=[nltk.word_tokenize(sent) for sent in text]
print(tokens)
from nltk.corpus import stopwords
def convert_to_lowercase(text):
    return [s.lower() for s in text]
def remove_punctuation_and_numbers(text):
    translator = str.maketrans('', '', string.punctuation + string.digits)
    return [s.translate(translator) for s in text]
def tokenize_words(text):
    return [nltk.word_tokenize(s) for s in text]
def remove_stopwords_from_tokens(tokenized_text, language='english'):
    stop_words = set(stopwords.words(language))
    return [[word for word in tokens if word not in stop_words] for tokens in t
def add_start_end_tokens(tokenized_text, start_token='<s>', end_token='</s>'):
    return [[start_token] + tokens + [end_token] for tokens in tokenized_text]
text_lower = convert_to_lowercase(text)
print("1. Lowercase:", text_lower)
text_no_punc_nums = remove_punctuation_and_numbers(text_lower)
print("2. No Punctuation/Numbers:", text_no_punc_nums)
text_tokenized = tokenize_words(text_no_punc_nums)
print("3. Tokenized:", text_tokenized)
text_no_stopwords = remove_stopwords_from_tokens(text_tokenized)
print("4. No Stopwords:", text_no_stopwords)
text_final_processed = add_start_end_tokens(text_no_stopwords)
print("5. Final Processed:", text_final_processed)
unigrams = []
for sentence_tokens in text_final_processed:
    for token in sentence_tokens:
        unigrams.append(token)
unigram_counts = collections.Counter(unigrams)
print("Unigram Counts:", unigram_counts)
bigrams = []
for sentence_tokens in text_final_processed:
```

```

    for i in range(len(sentence_tokens) - 1):
        bigrams.append((sentence_tokens[i], sentence_tokens[i+1]))
bigram_counts = collections.Counter(bigrams)
print("Bigram Counts:", bigram_counts)
trigrams = []
for sentence_tokens in text_final_processed:
    for i in range(len(sentence_tokens) - 2):
        trigrams.append((sentence_tokens[i], sentence_tokens[i+1], sentence_tok
trigram_counts = collections.Counter(trigrams)
print("Trigram Counts:", trigram_counts)
bigram_probabilities = {}
for (word1, word2), count in bigram_counts.items():
    #  $P(\text{word2} \mid \text{word1}) = \text{Count}(\text{word1}, \text{word2}) / \text{Count}(\text{word1})$ 
    probability = count / unigram_counts[word1]
    bigram_probabilities[(word1, word2)] = probability
print("Bigram Probabilities:")
for bigram, prob in bigram_probabilities.items():
    print(f"{bigram}: {prob:.4f}")
trigram_probabilities = {}
for (word1, word2, word3), count in trigram_counts.items():
    if (word1, word2) in bigram_counts:
        probability = count / bigram_counts[(word1, word2)]
        trigram_probabilities[(word1, word2, word3)] = probability
    else:
        trigram_probabilities[(word1, word2, word3)] = 0.0 # Or some small epsi
print("Trigram Probabilities:")
for trigram, prob in trigram_probabilities.items():
    print(f"{trigram}: {prob:.4f}")
unigram_df = pd.DataFrame(unigram_counts.items(), columns=['Unigram', 'Count'])
unigram_df = unigram_df.sort_values(by='Count', ascending=False).reset_index(dr
print("\n--- Unigram Counts ---")
print(unigram_df.to_string(index=False))
bigram_counts_df = pd.DataFrame([(bigram[0], bigram[1], count) for bigram, cour
    columns=['Word1', 'Word2', 'Count'])
bigram_counts_df = bigram_counts_df.sort_values(by='Count', ascending=False).re
print("\n--- Bigram Counts ---")
print(bigram_counts_df.to_string(index=False))
trigram_counts_df = pd.DataFrame([(trigram[0], trigram[1], trigram[2], count) f
    columns=['Word1', 'Word2', 'Word3', 'Count'])
trigram_counts_df = trigram_counts_df.sort_values(by='Count', ascending=False).
print("\n--- Trigram Counts ---")
print(trigram_counts_df.to_string(index=False))
bigram_probabilities_df = pd.DataFrame([{'Word1': bigram[0], 'Word2': bigram[1]
bigram_probabilities_df = bigram_probabilities_df.sort_values(by='Probability',
print("\n--- Bigram Probabilities ---")
print(bigram_probabilities_df.to_string(index=False))
trigram_probabilities_df = pd.DataFrame([{'Word1': trigram[0], 'Word2': trigran
trigram_probabilities_df = trigram_probabilities_df.sort_values(by='Probability
print("\n--- Trigram Probabilities ---")
print(trigram_probabilities_df.to_string(index=False))
V = len(unigram_df)

```

```
print(f"Vocabulary size (V): {V}")
smoothed_bigram_probabilities = {}
for (word1, word2), count_bigram in bigram_counts.items():
    count_word1 = unigram_counts.get(word1, 0) # Get count of word1. Should alw
    smoothed_prob = (count_bigram + 1) / (count_word1 + V)
    smoothed_bigram_probabilities[(word1, word2)] = smoothed_prob
print("Smoothed Bigram Probabilities (Add-one Smoothing):")
for bigram, prob in smoothed_bigram_probabilities.items():
    print(f"{bigram}: {prob:.4f}")
smoothed_bigram_probabilities_df = pd.DataFrame([{'Word1': bigram[0], 'Word2':
smoothed_bigram_probabilities_df = smoothed_bigram_probabilities_df.sort_values
print("\n--- Smoothed Bigram Probabilities (DataFrame) ---")
print(smoothed_bigram_probabilities_df.to_string(index=False))
smoothed_trigram_probabilities = {}
for (word1, word2, word3), count_trigram in trigram_counts.items():
    count_bigram_context = bigram_counts.get((word1, word2), 0) # Count of the
    smoothed_prob = (count_trigram + 1) / (count_bigram_context + V)
    smoothed_trigram_probabilities[(word1, word2, word3)] = smoothed_prob
print("Smoothed Trigram Probabilities (Add-one Smoothing):")
for trigram, prob in smoothed_trigram_probabilities.items():
    print(f"{trigram}: {prob:.4f}")
smoothed_trigram_probabilities_df = pd.DataFrame([{'Word1': trigram[0], 'Word2'
smoothed_trigram_probabilities_df = smoothed_trigram_probabilities_df.sort_val
print("\n--- Smoothed Trigram Probabilities (DataFrame) ---")
print(smoothed_trigram_probabilities_df.to_string(index=False))
```

```

    <s>      iam      0.333333
    iam      working  0.333333
    professor </s>    0.250000
    university </s>  0.250000
    sr university  0.250000
    working professor 0.222222
    working      sr   0.222222
Smoothed Trigram Probabilities (Add-one Smoothing):
(' <s>', 'iam', 'working'): 0.3333
('iam', 'working', 'professor'): 0.2222
('working', 'professor', ' </s>'): 0.2500
('iam', 'working', 'sr'): 0.2222
('working', 'sr', 'university'): 0.2500
('sr', 'university', ' </s>'): 0.2500

--- Smoothed Trigram Probabilities (DataFrame) ---
  Word1      Word2      Word3  Smoothed Probability
  <s>      iam      working      0.333333
working professor      </s>      0.250000
  sr university      </s>      0.250000
working      sr university  0.250000
  iam      working professor  0.222222
  iam      working      sr    0.222222
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

TASK 02 : TESTING THE 5 SENTENCES

```

test_sentences = [
    'This is a test sentence.',
    'Another example sentence.',
    'How are you?',
    'I am learning NLP.',
    'The quick brown fox jumps over the lazy dog.',
    'University professors teach many students.'
]
test_sentences_lower = convert_to_lowercase(test_sentences)
print("1. Lowercase:", test_sentences_lower)
test_sentences_no_punc_nums = remove_punctuation_and_numbers(test_sentences_lower)
print("2. No Punctuation/Numbers:", test_sentences_no_punc_nums)
test_sentences_tokenized = tokenize_words(test_sentences_no_punc_nums)
print("3. Tokenized:", test_sentences_tokenized)
test_sentences_no_stopwords = remove_stopwords_from_tokens(test_sentences_tokenized)
print("4. No Stopwords:", test_sentences_no_stopwords)
test_sentences_final_processed = add_start_end_tokens(test_sentences_no_stopwords)
print("5. Final Processed:", test_sentences_final_processed)
total_unigram_count = sum(unigram_counts.values())
print(f"Total number of unigrams (N) in training data: {total_unigram_count}")

```

```

smoothed_unigram_probabilities = {}
for word, count in unigram_counts.items():
    smoothed_prob = (count + 1) / (total_unigram_count + V)
    smoothed_unigram_probabilities[word] = smoothed_prob
print("Smoothed Unigram Probabilities (Add-one Smoothing):")
for word, prob in smoothed_unigram_probabilities.items():
    print(f"{word}: {prob:.4f}")
def calculate_unigram_probability(sentence_tokens, smoothed_unigram_probabiliti
    sentence_probability = 1.0
    oov_probability = 1 / (total_unigram_count + V)
    for word in sentence_tokens:
        prob = smoothed_unigram_probabilities.get(word, oov_probability)
        sentence_probability *= prob
    return sentence_probability
example_sentence = test_sentences_final_processed[0] # ['<s>', 'test', 'sentenc
sentence_prob = calculate_unigram_probability(example_sentence,
                                              smoothed_unigram_probabilities,
                                              total_unigram_count,
                                              V)
print(f"Example Sentence: {example_sentence}")
print(f"Smoothed Unigram Model Probability: {sentence_prob:.8f}")
example_sentence_with_oov = test_sentences_final_processed[4] # ['<s>', 'quick'
sentence_prob_oov = calculate_unigram_probability(example_sentence_with_oov,
                                                  smoothed_unigram_probabilitie
                                                  total_unigram_count,
                                                  V)
print(f"Example Sentence with OOV: {example_sentence_with_oov}")
print(f"Smoothed Unigram Model Probability (with OOV): {sentence_prob_oov:.8f}")
def calculate_bigram_probability(sentence_tokens, smoothed_unigram_probabilitie
    sentence_probability = 1.0
    oov_unigram_prob = 1 / (total_unigram_count + V)
    if not sentence_tokens:
        return 0.0 # Return 0 for an empty sentence
    first_word = sentence_tokens[0]
    sentence_probability *= smoothed_unigram_probabilities.get(first_word, oov_
    for i in range(1, len(sentence_tokens)):
        current_word = sentence_tokens[i]
        previous_word = sentence_tokens[i-1]
        bigram_prob = smoothed_bigram_probabilities.get((previous_word, current
        if bigram_prob is None:
            count_previous_word = unigram_counts.get(previous_word, 0)
            bigram_prob = 1 / (count_previous_word + V)
        sentence_probability *= bigram_prob
    return sentence_probability
example_sentence_bigram = test_sentences_final_processed[0] # ['<s>', 'test', '
sentence_prob_bigram = calculate_bigram_probability(
    example_sentence_bigram,
    smoothed_unigram_probabilities,
    smoothed_bigram_probabilities,
    unigram_counts,
    total_unigram_count,

```

```

V
)
print(f"\nExample Sentence for Bigram Model: {example_sentence_bigram}")
print(f"Smoothed Bigram Model Probability: {sentence_prob_bigram:.8f}")
example_sentence_bigram_oov = test_sentences_final_processed[4] # ['<s>', 'quic
sentence_prob_bigram_oov = calculate_bigram_probability(
    example_sentence_bigram_oov,
    smoothed_unigram_probabilities,
    smoothed_bigram_probabilities,
    unigram_counts,
    total_unigram_count,
    V
)
print(f"Example Sentence with OOV for Bigram Model: {example_sentence_bigram_oc
print(f"Smoothed Bigram Model Probability (with OOV): {sentence_prob_bigram_oov
empty_sentence = []
sentence_prob_empty = calculate_bigram_probability(
    empty_sentence,
    smoothed_unigram_probabilities,
    smoothed_bigram_probabilities,
    unigram_counts,
    total_unigram_count,
    V
)
print(f"Example Empty Sentence: {empty_sentence}")
print(f"Smoothed Bigram Model Probability (empty sentence): {sentence_prob_empt
def calculate_trigram_probability(sentence_tokens, smoothed_unigram_probabiliti
    sentence_probability = 1.0
    oov_unigram_prob = 1 / (total_unigram_count + V)
    if not sentence_tokens:
        return 0.0 # Return 0 for an empty sentence
    first_word = sentence_tokens[0]
    sentence_probability *= smoothed_unigram_probabilities.get(first_word, oov_
    if len(sentence_tokens) >= 2:
        second_word = sentence_tokens[1]
        previous_word_for_bigram = first_word
        bigram_prob = smoothed_bigram_probabilities.get((previous_word_for_bigr
        if bigram_prob is None:
            count_previous_word = unigram_counts.get(previous_word_for_bigram,
            bigram_prob = 1 / (count_previous_word + V)
        sentence_probability *= bigram_prob
    for i in range(2, len(sentence_tokens)):
        current_word = sentence_tokens[i]
        prev_word1 = sentence_tokens[i-2] # w_i-2
        prev_word2 = sentence_tokens[i-1] # w_i-1
        trigram_prob = smoothed_trigram_probabilities.get((prev_word1, prev_wor
        if trigram_prob is None:
            count_bigram_context = bigram_counts.get((prev_word1, prev_word2),
            trigram_prob = 1 / (count_bigram_context + V)
        sentence_probability *= trigram_prob
    return sentence_probability

```

```

example_sentence_trigram = test_sentences_final_processed[0] # ['<s>', 'test',
sentence_prob_trigram = calculate_trigram_probability(
    example_sentence_trigram,
    smoothed_unigram_probabilities,
    smoothed_bigram_probabilities,
    smoothed_trigram_probabilities,
    unigram_counts,
    bigram_counts,
    total_unigram_count,
    V
)
print(f"\nExample Sentence for Trigram Model: {example_sentence_trigram}")
print(f"Smoothed Trigram Model Probability: {sentence_prob_trigram:.8f}")
example_sentence_trigram_oov = test_sentences_final_processed[4] # ['<s>', 'qui
sentence_prob_trigram_oov = calculate_trigram_probability(
    example_sentence_trigram_oov,
    smoothed_unigram_probabilities,
    smoothed_bigram_probabilities,
    smoothed_trigram_probabilities,
    unigram_counts,
    bigram_counts,
    total_unigram_count,
    V
)
print(f"Example Sentence with OOV for Trigram Model: {example_sentence_trigram_oov}")
print(f"Smoothed Trigram Model Probability (with OOV): {sentence_prob_trigram_oov:.8f}")
empty_sentence = []
sentence_prob_empty_trigram = calculate_trigram_probability(
    empty_sentence,
    smoothed_unigram_probabilities,
    smoothed_bigram_probabilities,
    smoothed_trigram_probabilities,
    unigram_counts,
    bigram_counts,
    total_unigram_count,
    V
)
print(f"Example Empty Sentence: {empty_sentence}")
print(f"Smoothed Trigram Model Probability (empty sentence): {sentence_prob_empty_trigram:.8f}")
print("\n--- Sentence Probabilities --- ")
for i, original_sentence in enumerate(test_sentences):
    preprocessed_sentence = test_sentences_final_processed[i]
    print(f"\nOriginal Sentence: {original_sentence}")
    print(f"Preprocessed Tokens: {preprocessed_sentence}")
    unigram_prob = calculate_unigram_probability(
        preprocessed_sentence,
        smoothed_unigram_probabilities,
        total_unigram_count,
        V
    )
    print(f"Smoothed Unigram Model Probability: {unigram_prob:.8f}")

```

```

bigram_prob = calculate_bigram_probability(
    preprocessed_sentence,
    smoothed_unigram_probabilities,
    smoothed_bigram_probabilities,
    unigram_counts,
    total_unigram_count,
    V
)
print(f" Smoothed Bigram Model Probability: {bigram_prob:.8f}")
trigram_prob = calculate_trigram_probability(
    preprocessed_sentence,
    smoothed_unigram_probabilities,
    smoothed_bigram_probabilities,
    smoothed_trigram_probabilities,
    unigram_counts,
    bigram_counts,
    total_unigram_count,
    V
)
print(f" Smoothed Trigram Model Probability: {trigram_prob:.8f}")

```

university: 0.1111

Example Sentence: ['<s>', 'test', 'sentence', '</s>']

Smoothed Unigram Model Probability: 0.00008573

Example Sentence with OOV: ['<s>', 'quick', 'brown', 'fox', 'jumps', 'lazy', ' '

Smoothed Unigram Model Probability (with OOV): 0.00000000

Example Sentence for Bigram Model: ['<s>', 'test', 'sentence', '</s>']

Smoothed Bigram Model Probability: 0.00037793

Example Sentence with OOV for Bigram Model: ['<s>', 'quick', 'brown', 'fox', ' '

Smoothed Bigram Model Probability (with OOV): 0.00000016

Example Empty Sentence: []

Smoothed Bigram Model Probability (empty sentence): 0.00000000

Example Sentence for Trigram Model: ['<s>', 'test', 'sentence', '</s>']

Smoothed Trigram Model Probability: 0.00037793

Example Sentence with OOV for Trigram Model: ['<s>', 'quick', 'brown', 'fox', ' '

Smoothed Trigram Model Probability (with OOV): 0.00000016

Example Empty Sentence: []

Smoothed Trigram Model Probability (empty sentence): 0.00000000

Original Sentence: How are you?

Preprocessed Tokens: ['<s>', '</s>']

Smoothed Unigram Model Probability: 0.02777778

Smoothed Bigram Model Probability: 0.01851852

Smoothed Trigram Model Probability: 0.01851852

Original Sentence: I am learning NLP.

Preprocessed Tokens: ['<s>', 'learning', 'nlp', '</s>']

Smoothed Unigram Model Probability: 0.00008573

Smoothed Bigram Model Probability: 0.00037793

Smoothed Trigram Model Probability: 0.00037793

Original Sentence: The quick brown fox jumps over the lazy dog.

Preprocessed Tokens: ['<s>', 'quick', 'brown', 'fox', 'jumps', 'lazy', 'dog',

Smoothed Unigram Model Probability: 0.00000000

Smoothed Bigram Model Probability: 0.00000016

Smoothed Trigram Model Probability: 0.00000016

Original Sentence: University professors teach many students.

Preprocessed Tokens: ['<s>', 'university', 'professors', 'teach', 'many', 'stu

Smoothed Unigram Model Probability: 0.00000003

Smoothed Bigram Model Probability: 0.00000096

Smoothed Trigram Model Probability: 0.00000110

TASK 03 : APPLICATION PERPLEXITY TO THE DATA

```
import math
def calculate_perplexity(sentence_tokens, sentence_probability):
    effective_tokens = [token for token in sentence_tokens if token not in ['<s>']
    N = len(effective_tokens)
    if sentence_probability == 0:
        return float('inf')
    if N == 0:
        return 1.0
    perplexity = math.exp(-(math.log(sentence_probability) / N))
    return perplexity
print("Defined calculate_perplexity function.")
unigram_perplexities = []
print("\n--- Unigram Perplexities ---")
for i, original_sentence in enumerate(test_sentences):
    preprocessed_sentence = test_sentences_final_processed[i]
    unigram_prob = calculate_unigram_probability(
        preprocessed_sentence,
        smoothed_unigram_probabilities,
        total_unigram_count,
        V
    )
    unigram_perplexity = calculate_perplexity(preprocessed_sentence, unigram_prob)
    unigram_perplexities.append(unigram_perplexity)
    print(f"\nOriginal Sentence: {original_sentence}")
    print(f"Preprocessed Tokens: {preprocessed_sentence}")
```

```

    print(f" Smoothed Unigram Model Perplexity: {unigram_perplexity:.4f}")
    bigram_perplexities = []
print("\n--- Bigram Perplexities ---")
for i, original_sentence in enumerate(test_sentences):
    preprocessed_sentence = test_sentences_final_processed[i]
    bigram_prob = calculate_bigram_probability(
        preprocessed_sentence,
        smoothed_unigram_probabilities,
        smoothed_bigram_probabilities,
        unigram_counts,
        total_unigram_count,
        V
    )
    bigram_perplexity = calculate_perplexity(preprocessed_sentence, bigram_prob)
    bigram_perplexities.append(bigram_perplexity)
    print(f"\nOriginal Sentence: {original_sentence}")
    print(f"Preprocessed Tokens: {preprocessed_sentence}")
    print(f" Smoothed Bigram Model Perplexity: {bigram_perplexity:.4f}")
    trigram_perplexities = []
print("\n--- Trigram Perplexities ---")
for i, original_sentence in enumerate(test_sentences):
    preprocessed_sentence = test_sentences_final_processed[i]
    trigram_prob = calculate_trigram_probability(
        preprocessed_sentence,
        smoothed_unigram_probabilities,
        smoothed_bigram_probabilities,
        smoothed_trigram_probabilities,
        unigram_counts,
        bigram_counts,
        total_unigram_count,
        V
    )
    trigram_perplexity = calculate_perplexity(preprocessed_sentence, trigram_prob)
    trigram_perplexities.append(trigram_perplexity)
    print(f"\nOriginal Sentence: {original_sentence}")
    print(f"Preprocessed Tokens: {preprocessed_sentence}")
    print(f" Smoothed Trigram Model Perplexity: {trigram_perplexity:.4f}")
    perplexity_data = []
for i, original_sentence in enumerate(test_sentences):
    perplexity_data.append({
        'Original Sentence': original_sentence,
        'Preprocessed Tokens': test_sentences_final_processed[i],
        'Unigram Perplexity': unigram_perplexities[i],
        'Bigram Perplexity': bigram_perplexities[i],
        'Trigram Perplexity': trigram_perplexities[i]
    })
perplexity_df = pd.DataFrame(perplexity_data)
print("\n--- Perplexity Comparison Across Models ---")
print(perplexity_df.to_string(index=False))
print("\n--- Perplexity Comparison Across Models ---")
print(perplexity_df.to_string(index=False))

```

```
Preprocessed Tokens: ['<s>', 'I', 'am', 'learning', 'nlp', '</s>']
```

Smoothed Bigram Model Perplexity: 1.0000

Original Sentence: I am learning NLP.

Preprocessed Tokens: ['<s>', 'learning', 'nlp', '</s>']

Smoothed Bigram Model Perplexity: 51.4393

Original Sentence: The quick brown fox jumps over the lazy dog.

Preprocessed Tokens: ['<s>', 'quick', 'brown', 'fox', 'jumps', 'lazy', 'dog',

Smoothed Bigram Model Perplexity: 13.6091

Original Sentence: University professors teach many students.

Preprocessed Tokens: ['<s>', 'university', 'professors', 'teach', 'many', 'stu

Smoothed Bigram Model Perplexity: 15.9652

--- Trigram Perplexities ---

University professors teach many students. [<s>, university, professors, tea